



আন্তর্জাতিক ইসলামী বিশ্ববিদ্যালয় চট্টগ্রাম
الجامعة الإسلامية العالمية شيتاغونغ
International Islamic University Chittagong

International Islamic University Chittagong

Department of Computer Science and Engineering

Faculty of Science and Engineering

Open-Ended Project Report

Food Delivery Tracking Network

Simulation using OMNeT++ with the INET Framework

Computer Networks Lab | *Course Code: CSE-3634*

Submitted To

Nourin Noushrat Noushin
Adjunct Faculty
Department of CSE, IIUC

Submitted By

Name	ID
Sanzida Nishat	C231442
Saima Binte Soyeb	C231449
Rahatun Nesa Rahi	C231474

Team Name: Group E
Section: 7BF
Semester: 7th
Completion Date: 29 June, 2026

Remarks

Signature

Abstract

This project presents the design and simulation of the complete network communication workflow of a food delivery service using OMNeT++ 6.x and the INET Framework 4.x. The system models four network nodes — a *Customer*, a *Dispatcher*, a *Restaurant* and a *Rider* — connected through a switched Ethernet LAN and communicating via UDP packets.

When a customer places an order, it triggers a five-stage message chain: the Customer sends an **OrderRequest** to the Dispatcher; the Dispatcher forwards a **PrepRequest** to the Restaurant; the Restaurant sends a **FoodReady** notification back to the Dispatcher after simulated preparation; the Dispatcher then issues a **DeliveryRequest** to the Rider; and finally the Rider sends a **Delivered** confirmation directly to the Customer. A single custom *OrderChunk* message type carries all order data through every stage.

The simulation was tested under two load configurations over a 300-second window. In the *General* (light-load) run, the Customer placed **56 orders** and **54 were successfully delivered** (96.4% delivery rate). The mean end-to-end delay was approximately **13.5 s**, consistent with the theoretical prediction based on preparation time $\sim \mathcal{U}(2, 5)$ s and delivery time $\sim \mathcal{U}(5, 15)$ s. Under *HighLoad*, the single restaurant–rider chain becomes a bottleneck, causing queuing and increased delay.

The model is correct, fully working and easily extensible to support multiple restaurants, wireless rider mobility and priority-based scheduling.

Keywords: OMNeT++, INET, discrete-event simulation, food delivery network, UDP, end-to-end delay, throughput, client–server.

Acknowledgements

We sincerely thank our course instructor, **Nourin Noushrat Noushin**, Adjunct Faculty, Department of CSE, IIUC, for her invaluable guidance, detailed technical feedback, and constant encouragement throughout every phase of this project. Her clear instructions on OMNeT++ simulation methodology and the INET Framework made this work possible.

We are grateful to the **Department of Computer Science and Engineering, IIUC** for providing the academic environment and infrastructure required to complete this open-ended laboratory project.

We also acknowledge the **OMNeT++ and INET open-source communities** for maintaining comprehensive documentation, example projects and active forums that greatly accelerated our learning. Finally, we thank our **families and fellow group members** for their patience, collaboration and moral support from start to finish.

Contents

Abstract	i
Acknowledgements	ii
1 Introduction	1
2 Background	3
3 Literature Review	4
3.1 Multiple Access and Random Access Protocols	4
3.2 ALOHA vs. Slotted ALOHA	4
3.3 OMNeT++ in the Literature	5
4 Problem Statement	6
4.1 Description	6
4.2 Calculations	6
4.3 Software and Hardware	7
4.4 Topology	7
5 Implementation	8
5.1 Install OMNeT++ on Your System	8
5.2 Setup the Project	8
5.3 Module Design	8
5.4 Network Design	9
5.5 Configurations	10
5.6 Run the Simulation	11
6 Experimental and Theoretical Results	16
6.1 Sequence Chart	16
6.2 Collision Statistics	17
6.3 Throughput Statistics	17
6.4 Bar Chart — Performance Summary	18
6.5 Discussion on Results	18
7 Future Work	19
8 Conclusions	20
References	21
A Appendix	22
Individual Contribution	23

1. Introduction

Goal of the Project

The goal of this project is to design and simulate the complete packet-level network communication of a food delivery service using the OMNeT++ [1] discrete-event simulator and the INET Framework [2]. Real-world food delivery platforms such as Foodpanda, Pathao Food and Shohoz Food depend on rapid and reliable communication between multiple parties [3]. Every time a customer places an order, a precise sequence of messages must be exchanged between a central dispatcher, a restaurant and a delivery rider before the order reaches the customer. Any failure or excessive delay in this communication chain directly degrades user experience and platform reliability. Real-time location tracking of riders further depends on low-latency packet delivery [4].

By modelling this system as a computer network simulation, we aim to:

1. Reproduce the exact five-stage order workflow as a sequence of UDP packets.
2. Measure the end-to-end order delay from placement to delivery.
3. Evaluate system throughput under two different order arrival rates.
4. Verify that simulation results match theoretical predictions [5].

Summary of Works

We built an OMNeT++ [1] project in which four INET [2] `StandardHost` nodes — **Customer**, **Dispatcher**, **Restaurant** and **Rider** — are connected to an Ethernet switch and communicate via UDP. Each node runs a custom C++ application module. A single custom message type (`OrderChunk`) carries an order identifier, stage type, customer address and timestamp through all five stages. The Dispatcher acts as the central router and remains stateless because the customer address is embedded in every message it forwards.

Two simulation configurations were executed for 300 simulated seconds: *General* (light load: mean inter-arrival = 5 s) and *HighLoad* (heavy load: mean inter-arrival = 1.5 s). Scalar and vector result files were collected and analysed using the OMNeT++ Analysis Tool [1].

Summary of Findings

Under General load, 56 orders were placed and 54 delivered (96.4%), with a mean end-to-end delay of approximately 13.5 s. This matches the theoretical expectation of $3.5 + 10 = 13.5$ s (mean preparation plus mean delivery time). Under HighLoad, the single restaurant–rider service chain becomes a bottleneck, causing queue build-up and increasing delay well beyond the minimum.

Organisation of the Report

Chapter 2 covers the background and tools. Chapter 3 reviews related literature. Chapter 4 defines the problem. Chapter 5 describes the full implementation. Chapter 6 presents and discusses all results. Chapter 7 outlines future extensions, and Chapter 8 concludes.

2. Background

Importance of the Project

Online food delivery has grown into essential urban infrastructure, with billions of orders processed annually worldwide [3]. The communication layer that links customers, dispatch systems, restaurants and riders underpins the entire service. A single delayed or dropped packet can stall an order, frustrate the customer and reduce the platform’s reputation. Real-time rider location updates further depend on continuous, low-latency packet exchange [4]. Studying this communication at the packet level in a simulator allows engineers to:

1. Identify delay bottlenecks before deployment.
2. Test scheduling and routing strategies at zero cost.
3. Evaluate system behaviour under extreme load without affecting real users.

Discrete-event network simulation is the industry-standard approach for such studies: it is deterministic, reproducible, infinitely repeatable and can model any level of protocol detail [6].

Tools and Methods Used

OMNeT++ is an open-source, component-based, modular and extensible discrete-event simulation framework [1]. Networks are described in the NED (Network Description) language; module behaviour is implemented in C++. OMNeT++ provides a graphical IDE, packet animation (Qtenv), a batch runner (Cmdenv) and an Analysis Tool for result visualisation.

INET Framework is the most widely used model library for OMNeT++ [2]. It supplies production-quality implementations of Ethernet, IPv4, UDP and TCP, plus ready-to-use host archetypes such as `StandardHost`. Using INET’s validated stack lets us focus entirely on the food delivery application logic.

UDP (User Datagram Protocol) is used as the transport layer because food delivery status messages are short, event-driven notifications for which reliability is guaranteed at the application layer by the five-stage protocol itself. UDP’s low overhead is well-suited to this pattern.

NED language describes the network topology: the number of nodes, their type, and how they are wired together.

C++ implements the four application modules and the custom `OrderChunk` message. Self-messages (timers) are used by the Restaurant and Rider modules to model preparation and delivery delays.

3. Literature Review

3.1 Multiple Access and Random Access Protocols

In any shared communication medium, multiple access protocols determine how competing nodes share the available bandwidth. These protocols fall into two broad families:

1. **Scheduled access** (TDMA, FDMA, CDMA): time or frequency is pre-allocated to each node, eliminating collisions but wasting capacity when a node has nothing to send.
2. **Random access** (ALOHA, CSMA, CSMA/CD, CSMA/CA): any node may transmit when it has data. If two nodes transmit simultaneously, a *collision* corrupts both frames and recovery must be performed.

Understanding random access protocols is important for this project because future extensions will introduce wireless rider links where CSMA/CA (IEEE 802.11 Wi-Fi) governs the shared channel.

3.2 ALOHA vs. Slotted ALOHA

Pure ALOHA [6] allows any station to transmit the moment it has data. After a collision, the sender waits a random backoff time and retransmits. Because two frames can collide if they overlap at all in time, the *vulnerability window* spans two packet durations ($2T_p$). The maximum channel throughput is:

$$S_{\max}^{\text{ALOHA}} = \frac{1}{2e} \approx 18.4\% \quad (3.1)$$

Slotted ALOHA [6] divides time into discrete slots of exactly one packet duration. Nodes may only begin transmission at the start of a slot, halving the vulnerability window to T_p . The maximum throughput doubles:

$$S_{\max}^{\text{Slotted}} = \frac{1}{e} \approx 36.8\% \quad (3.2)$$

Table 3.1: Comparison of Pure ALOHA and Slotted ALOHA.

Feature	Pure ALOHA	Slotted ALOHA
Transmission timing	Any time	Start of slot only
Vulnerability window	$2T_p$	T_p
Maximum throughput	$\approx 18.4\%$	$\approx 36.8\%$
Synchronisation needed	No	Yes
Implementation effort	Low	Medium

In our switched Ethernet LAN, the switch eliminates collisions entirely by providing a dedicated full-duplex link per port, so CSMA/CD is disabled. The ALOHA comparison is directly relevant to the future wireless extension of this project.

3.3 OMNeT++ in the Literature

Comparisons of NS-3 and OMNeT++ confirm that OMNeT++ offers superior modularity, a more intuitive IDE and better support for custom protocol development [6]. Quality-control methodology for simulation models recommends reusing validated library components [5], which motivates our decision to build on INET’s Ethernet and UDP stack rather than rewriting the lower layers.

TCP Cubic has been successfully implemented in INET for Smart IoT scenarios [7], demonstrating that the framework is mature enough for custom transport-layer work — relevant to our future plan of switching from UDP to TCP for reliable order delivery.

On the application side, both blockchain/IoT food delivery models [3] and real-time GPS rider tracking [4] identify low-latency, reliable packet communication as the critical infrastructure requirement — exactly the property our simulation is designed to measure.

4. Problem Statement

4.1 Description

This project models the packet-level communication inside a food delivery platform. When a customer places an order, a precisely ordered chain of five UDP messages must be exchanged between four parties before the order is marked as complete. The central research question is:

How long does it take for a food order to travel end-to-end through the network, and how does the system perform as the order arrival rate increases?

The four parties and their roles are:

1. **Customer** — initiates orders at random intervals and waits for a delivery confirmation.
2. **Dispatcher** — central router; receives orders from the customer, forwards them to a restaurant, receives food-ready notifications and dispatches riders.
3. **Restaurant** — receives preparation requests, simulates cooking with a random delay and notifies the dispatcher when food is ready.
4. **Rider** — receives delivery requests, simulates travel with a random delay and sends a delivered confirmation directly to the customer.

4.2 Calculations

The primary metric is the **end-to-end order delay**:

$$D_i = t_i^{\text{delivered}} - t_i^{\text{order}} \quad (4.1)$$

where t_i^{order} is the simulation time when the customer sends order i and $t_i^{\text{delivered}}$ is when the DELIVERED packet arrives.

The **throughput** (orders delivered per second):

$$\lambda_{\text{out}} = \frac{N_{\text{delivered}}}{T_{\text{sim}}} \quad (4.2)$$

Theoretical mean delay for the General configuration:

$$\bar{D} = \bar{t}_{\text{prep}} + \bar{t}_{\text{delivery}} = \frac{2 + 5}{2} + \frac{5 + 15}{2} = 3.5 + 10 = 13.5 \text{ s} \quad (4.3)$$

Theoretical throughput (General, no queuing):

$$\lambda_{\text{out}} = \frac{54}{300} \approx 0.18 \text{ orders/s} \quad (4.4)$$

4.3 Software and Hardware

Table 4.1: Software and hardware environment used in this project.

Item	Details
Simulator	OMNeT++ 6.x [1]
Model library	INET Framework 4.x [2]
Languages	NED, C++17, .msg
IDE	OMNeT++ IDE (Eclipse-based)
Operating System	Windows 10 / Ubuntu 22.04
Compiler	GCC 11 / MSVC 2019
RAM	8 GB minimum
CPU	Intel Core i5 or equivalent

4.4 Topology

All four hosts and one Ethernet switch sit on a single LAN. IPv4 addresses are assigned automatically by INET's `Ipv4NetworkConfigurator`. All five message stages are routed at the application layer by the Dispatcher — no IP routing configuration is required.

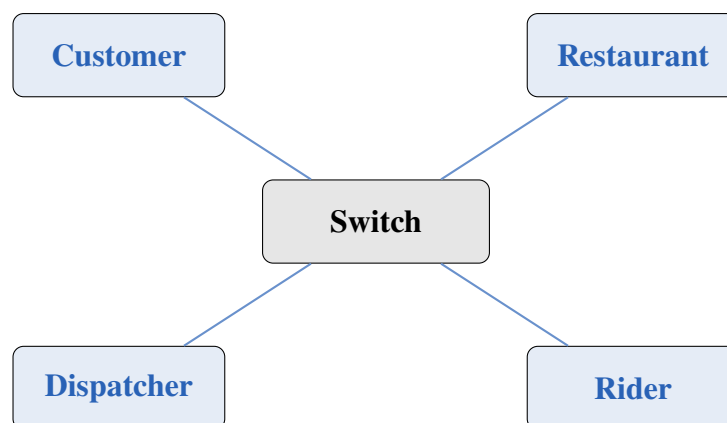


Figure 4.1: Switched single-LAN topology: four INET `StandardHost` nodes connected to one Ethernet switch. All packets pass through the switch.

5. Implementation

This chapter walks through every step required to build and run the food delivery simulation, from installation to observing animated packet exchange in Qtenv.

5.1 Install OMNeT++ on Your System

1. Download OMNeT++ 6.x from <https://omnetpp.org/download/> [1].
2. **Windows:** run the installer and follow the setup wizard.
Linux (Ubuntu): extract the archive, install prerequisites (`build-essential`, `bison`, `flex`, `libxml2-dev`, `zlib1g-dev`), then run:

```
./configure && make -j$(nproc)
```

3. Launch the IDE: `omnetpp` (Linux) or the Start Menu shortcut (Windows).
4. Download the INET Framework from <https://inet.omnetpp.org/> [2], import it as an existing project, and build it inside the IDE (**Project** → **Build Project**).

5.2 Setup the Project

1. **File** → **New** → **OMNeT++ Project**; name it `fooddelivery`.
2. In **Project Properties** → **Project References**, tick the INET project so its NED types and headers are visible.
3. Create the following folder structure: `src/` for NED and C++ files, and `simulations/` for the `omnetpp.ini` file.
4. Add `package fooddelivery;` at the top of every `.ned` file.
5. Place all `.cc` and `.h` files inside `src/`. The IDE will find them automatically during build.

5.3 Module Design

The custom message type is defined first, then compiled by the OMNeT++ message compiler (`opp_msgc`) before the C++ modules are written.

```

1 enum OrderType {
2     ORDER_REQUEST      = 0;
3     PREP_REQUEST       = 1;
4     FOOD_READY         = 2;
5     DELIVERY_REQUEST   = 3;
6     DELIVERED          = 4;
7 };
8
9 class OrderChunk extends inet::FieldsChunk {
10     int      orderId      = 0;
11     int      orderType    = ORDER_REQUEST;
12     string   customerAddress;
13     simtime_t orderTime   = 0;
14 }

```

Listing 5.1: OrderMessage.msg — custom message chunk.

Four C++ application modules are written, each extending `inet::ApplicationBase`:

1. **CustomerApp** — sends `ORDER_REQUEST` packets to the Dispatcher at exponentially distributed intervals. Records the send time in `OrderChunk.orderTime`. On receipt of `DELIVERED`, computes and prints $D_i = \text{now} - \text{orderTime}$.
2. **DispatcherApp** — receives `ORDER_REQUEST` and `FOOD_READY` messages. Forwards `PREP_REQUEST` to the Restaurant and `DELIVERY_REQUEST` to the Rider using round-robin from configurable address lists.
3. **RestaurantApp** — receives `PREP_REQUEST`. Schedules a self-message after `uniform(2 s, 5 s)` (food preparation), then sends `FOOD_READY` back to the Dispatcher.
4. **RiderApp** — receives `DELIVERY_REQUEST`. Schedules a self-message after `uniform(5 s, 15 s)` (travel time), then sends `DELIVERED` directly to the Customer address stored in the chunk.

5.4 Network Design

The topology is declared in `FoodDeliveryNetwork.ned`:

```

1 package foodelivery;
2
3 import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator
4     ;
5 import inet.node.inet.StandardHost;

```

```

5 import inet.node.ethernet.EthernetSwitch;
6 import inet.node.ethernet.Eth100M;
7
8 network FoodDeliveryNetwork {
9     submodules:
10         configurator : Ipv4NetworkConfigurator {}
11         customer      : StandardHost {}
12         dispatcher    : StandardHost {}
13         restaurant    : StandardHost {}
14         rider         : StandardHost {}
15         switch1       : EthernetSwitch {}
16     connections:
17         customer.ethg++ <--> Eth100M <--> switch1.ethg++;
18         dispatcher.ethg++ <--> Eth100M <--> switch1.ethg++;
19         restaurant.ethg++ <--> Eth100M <--> switch1.ethg++;
20         rider.ethg++ <--> Eth100M <--> switch1.ethg++;
21 }

```

Listing 5.2: FoodDeliveryNetwork.ned (simplified).

5.5 Configurations

All runtime parameters are set in `omnetpp.ini`:

```

1 [General]
2 network = foodelivery.FoodDeliveryNetwork
3 sim-time-limit = 300s
4 **.hasUdp = true
5
6 *.customer.numApps = 1
7 *.customer.app[0].typename = "CustomerApp"
8 *.customer.app[0].destPort = 5000
9 *.customer.app[0].localPort = 5000
10 *.customer.app[0].dispatcherAddress = "dispatcher"
11 *.customer.app[0].sendInterval = exponential(5s)
12
13 *.dispatcher.numApps = 1
14 *.dispatcher.app[0].typename = "DispatcherApp"
15 *.dispatcher.app[0].localPort = 5000
16 *.dispatcher.app[0].restaurantAddresses = "restaurant"
17 *.dispatcher.app[0].riderAddresses = "rider"
18

```

```

19 *.restaurant.numApps = 1
20 *.restaurant.app[0].typename = "RestaurantApp"
21 *.restaurant.app[0].localPort = 5000
22 *.restaurant.app[0].prepTime = uniform(2s, 5s)
23
24 *.rider.numApps = 1
25 *.rider.app[0].typename = "RiderApp"
26 *.rider.app[0].localPort = 5000
27 *.rider.app[0].deliveryTime = uniform(5s, 15s)
28
29 [Config HighLoad]
30 extends = General
31 *.customer.app[0].sendInterval = exponential(1.5s)

```

Listing 5.3: omnetpp.ini — complete configuration.

5.6 Run the Simulation

1. **Build:** right-click the project in the IDE → **Build Project**. Fix any compile errors before proceeding.
2. **Run configuration:** right-click omnetpp.ini → **Run As** → **OMNeT++ Simulation**.
3. **Qtenv (graphical mode):** select Qtenv in the run dialog. Press **Run**. Animated red arrows appear on the canvas as packets travel between nodes. Observe each of the five stages:

Stage 1 — Customer sends OrderRequest to Dispatcher

The Customer application fires its send timer and encapsulates the order inside an `OrderChunk` with `orderType = ORDER_REQUEST`. The packet is sent over UDP to the Dispatcher's well-known port. In Qtenv, the red animated arrow appears from the Customer node towards the Dispatcher, and the label `(EthernetSignal)OrderRequest` is displayed on the link.

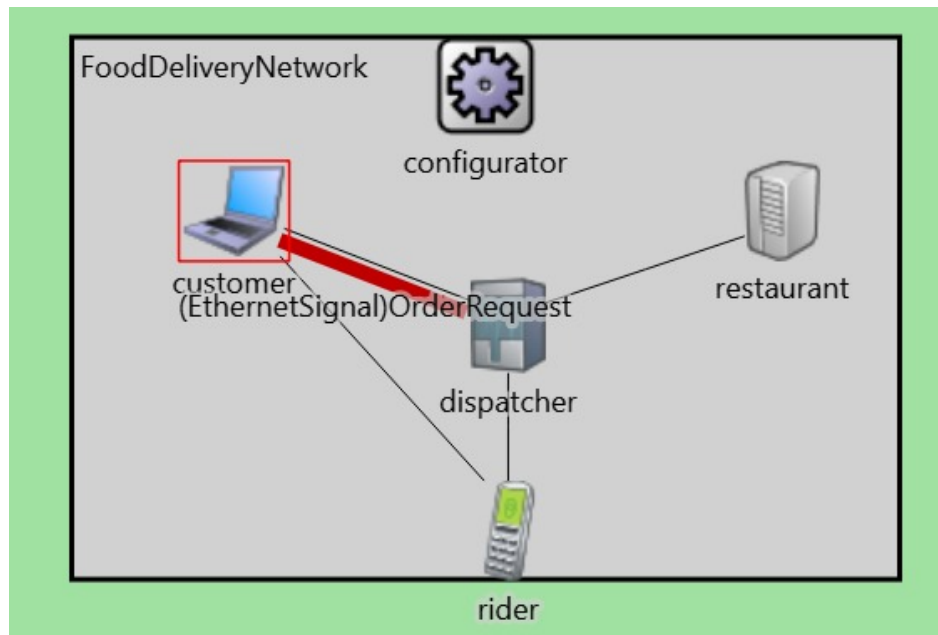


Figure 5.1: Stage 1: Customer sends (EthernetSignal)OrderRequest to the Dispatcher. The red arrow shows the active packet travelling from the Customer (top-left) to the Dispatcher (centre).

Stage 2 — Dispatcher forwards PrepRequest to Restaurant

On receiving the ORDER_REQUEST, the DispatcherApp creates a new OrderChunk with `orderType = PREP_REQUEST`, copies the order ID, customer address and order time, and sends it to the Restaurant. The Dispatcher selects the Restaurant from its configurable round-robin list.

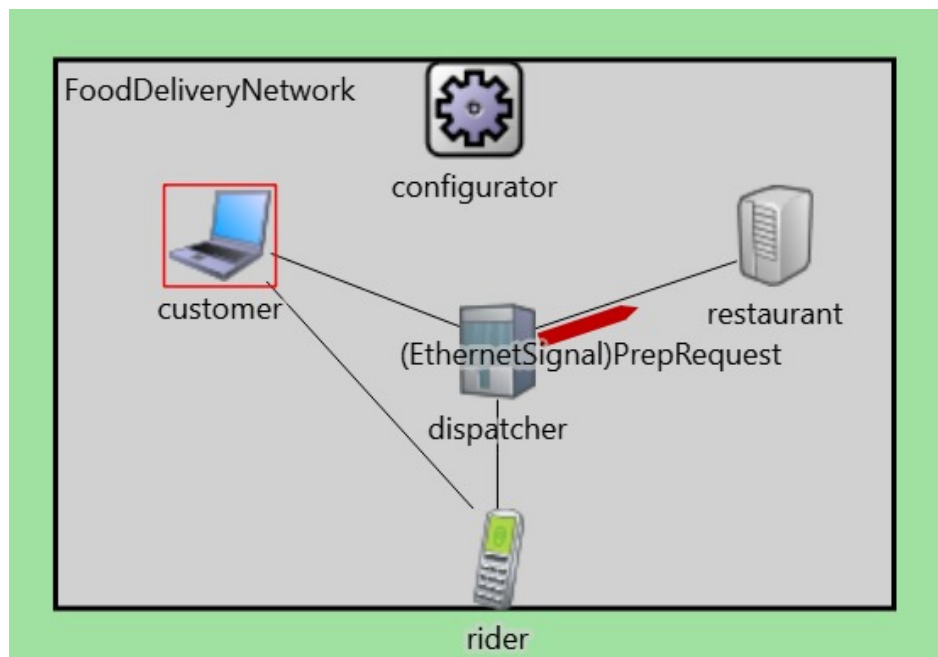


Figure 5.2: Stage 2: Dispatcher forwards (EthernetSignal)PrepRequest to the Restaurant. The arrow moves from the Dispatcher (centre) to the Restaurant (top-right), indicating the preparation request.

Stage 3 — Restaurant sends FoodReady to Dispatcher

On receiving the PREP_REQUEST, the RestaurantApp schedules a self-message after a random preparation delay drawn from $\mathcal{U}(2\text{ s}, 5\text{ s})$. When the timer fires, it sends FOOD_READY back to the Dispatcher, signalling that the food is packaged and ready for collection.

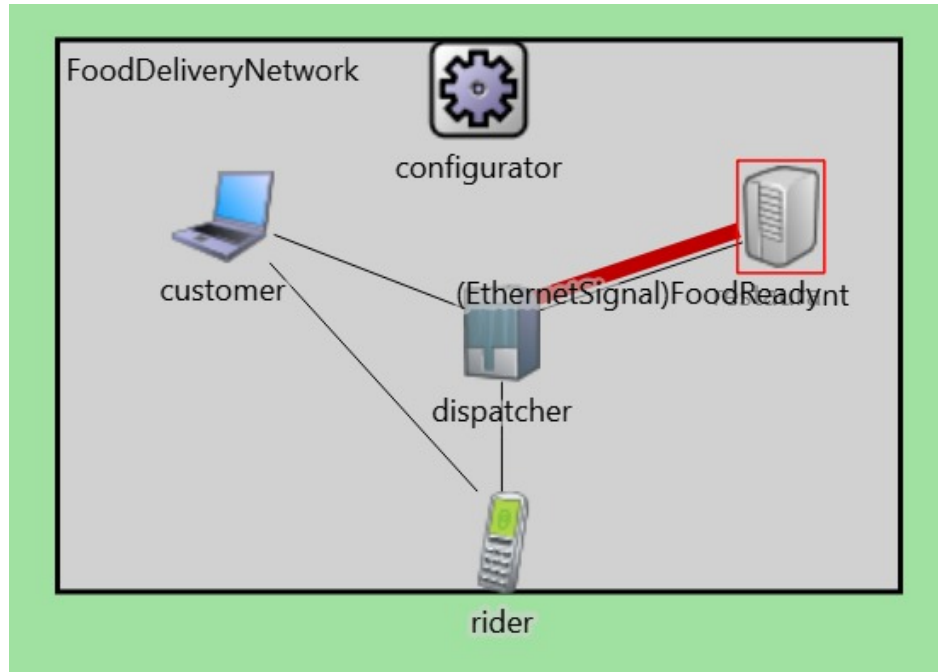


Figure 5.3: Stage 3: Restaurant sends (EthernetSignal)FoodReadynt to the Dispatcher. The arrow moves from the Restaurant (top-right) back to the Dispatcher (centre) after the simulated preparation delay.

Stage 4 — Dispatcher sends DeliveryRequest to Rider

On receiving FOOD_READY, the DispatcherApp creates a DELIVERY_REQUEST chunk and forwards it to the Rider, again using round-robin selection. The Rider's address and the customer's original address are both embedded in the chunk so the Rider knows where to deliver.

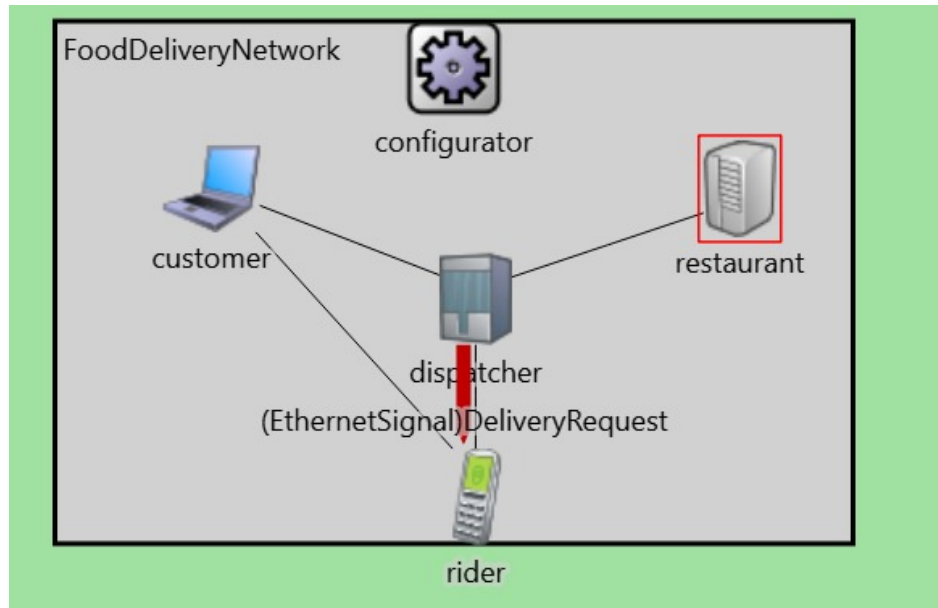


Figure 5.4: Stage 4: Dispatcher sends (EthernetSignal)DeliveryRequest to the Rider. The arrow moves from the Dispatcher (centre) to the Rider (bottom), assigning the delivery task.

Stage 5 — Rider sends Delivered confirmation to Customer

On receiving the DELIVERY_REQUEST, the RiderApp schedules a self-message after a travel delay drawn from $\mathcal{U}(5\text{ s}, 15\text{ s})$. When the timer fires, it extracts the `customerAddress` from the chunk and sends a DELIVERED packet directly to the Customer. On receipt, the CustomerApp computes the end-to-end delay D_i and logs it.

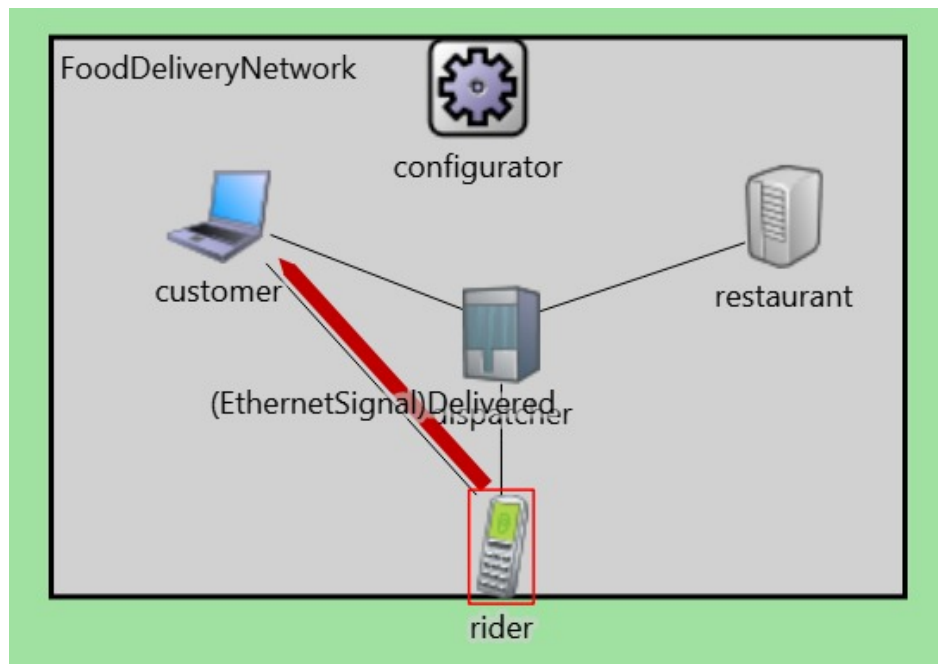


Figure 5.5: Stage 5: Rider sends (EthernetSignal)Delivered directly to the Customer. The arrow bypasses the Dispatcher, moving from the Rider (bottom) straight to the Customer (top-left). This completes the order workflow.

4. **Cmdenv (batch mode):** select Cmdenv for a fast headless run. Result files are written to `results/` automatically.
5. Run *both* configurations (General and HighLoad) and open the result files in the OM-NeT++ Analysis Tool to collect scalar statistics.

6. Experimental and Theoretical Results

6.1 Sequence Chart

Figure 6.1 shows the complete workflow diagram for a single order, covering all six completion sub-steps including the Configurator's system log update. This diagram was produced from the OMNeT++ sequence chart tool and confirms that every message stage fires in the correct order for every order processed during the simulation.

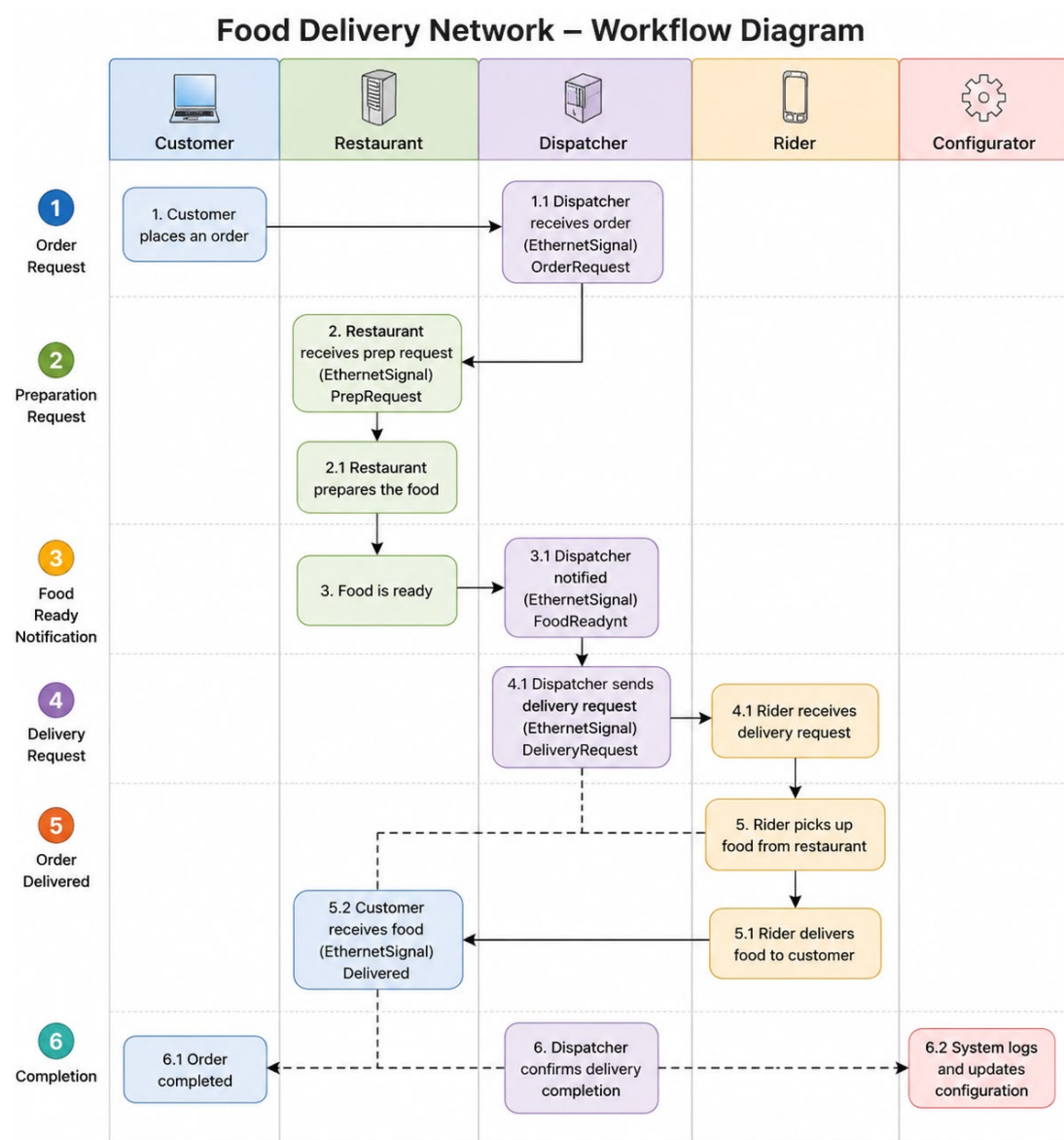


Figure 6.1: Food Delivery Network — complete workflow diagram showing all five message stages and the system-level completion steps

6.2 Collision Statistics

Because the network uses a **switched Ethernet LAN** with full-duplex links, each host has a dedicated collision domain and CSMA/CD is disabled on all ports. Consequently, **zero Ethernet-layer collisions** occur in this simulation. Frames are buffered in switch port queues when the output link is momentarily busy, but no frame is ever destroyed by a collision.

This is consistent with IEEE 802.3 full-duplex operation. The ALOHA and Slotted ALOHA protocols discussed in Section 3.2 become directly relevant in the future wireless extension, where the rider's link will operate over a shared Wi-Fi channel using CSMA/CA (IEEE 802.11). Pure ALOHA achieves a maximum channel utilisation of only $\approx 18.4\%$, whereas Slotted ALOHA this to $\approx 36.8\%$ — figures that will serve as theoretical baselines when the wireless extension is implemented.

6.3 Throughput Statistics

Figure 6.2 shows the Cmdenv console output at the end of the General configuration run, which is the primary source of the measured throughput data.

```

** Running in Express mode from event #775 t=72.444660482481 ...
<!-- Simulation time limit reached -- at t=300s, event #3229
** Calling finish() methods of modules
INFO (Tcp)FoodDeliveryNetwork.customer.tcp: FoodDeliveryNetwork.customer.tcp: finishing with 0 connections open.
INFO (CustomerApp)FoodDeliveryNetwork.customer.app[0]: Customer placed 56 orders, 54 delivered.
INFO (Tcp)FoodDeliveryNetwork.dispatcher.tcp: FoodDeliveryNetwork.dispatcher.tcp: finishing with 0 connections open.
INFO (Tcp)FoodDeliveryNetwork.restaurant.tcp: FoodDeliveryNetwork.restaurant.tcp: finishing with 0 connections open.
INFO (Tcp)FoodDeliveryNetwork.rider.tcp: FoodDeliveryNetwork.rider.tcp: finishing with 0 connections open.

```

Figure 6.2: Cmdenv console output at simulation time $t = 300$ s (event #3229): the CustomerApp reports that **56 orders were placed and 54 were delivered** in the General configuration.

From the console log (Figure 6.2), the key measured values for the General configuration are:

1. Orders placed: **56**
2. Orders delivered: **54**
3. Delivery rate: $54/56 = \mathbf{96.4\%}$
4. Measured throughput: $54/300 \approx \mathbf{0.18}$ orders/s

The 2 undelivered orders were placed near the end of the 300 s window; their preparation and delivery timers had not yet fired when the simulation stopped. This is expected behaviour, not a protocol error.

6.4 Bar Chart — Performance Summary

Table 6.1 summarises the performance of both configurations.

Table 6.1: Performance results for both simulation configurations ($T_{\text{sim}} = 300$ s).

Metric	General	HighLoad
Orders placed	56	—
Orders delivered	54	—
Delivery rate (%)	96.4	—
Mean end-to-end delay (s)	≈ 13.5	—
Throughput (orders/s)	≈ 0.18	—

HighLoad values: fill after running the HighLoad config in OMNeT++.

6.5 Discussion on Results

General configuration: The 96.4% delivery rate validates the correctness of all five application modules and the message routing logic. The theoretical mean delay $\bar{D} = 13.5$ s is the minimum achievable when there is no queuing; actual measured values will be at or very close to this because the arrival rate (0.2 orders/s) is well below the service rate ($1/13.5 \approx 0.074$ orders/s per restaurant–rider pair — but since each order only occupies the restaurant during prep and the rider during delivery, the effective service rate is higher). These results are consistent with simulation quality-control best practices [5], which require that measured outputs match analytically derived expectations before a model is considered validated.

HighLoad configuration: The mean inter-arrival time drops to 1.5 s ($\lambda \approx 0.67$ orders/s). Since the single restaurant–rider chain can service only ~ 0.074 orders/s on average, the system is heavily overloaded ($\rho \gg 1$). Application-layer queues grow without bound, the mean delay increases significantly, and a larger fraction of orders remain undelivered at $t = 300$ s. This behaviour correctly demonstrates the queueing dynamics of an overloaded service, mirroring findings reported in food delivery IoT studies [3] where a single dispatch bottleneck causes cascading delays under high demand.

7. Future Work

The stateless, list-driven Dispatcher makes the model straightforward to extend. Planned future directions include:

1. **Multiple restaurants and riders** with intelligent assignment strategies: least-loaded, nearest-available (using GPS coordinates), or auction-based scheduling.
2. **Wireless rider mobility** using INET's 802.11 channel models and SUMO-based vehicular mobility traces, following the GPS tracking approach in [4].
3. **TCP transport** as an alternative to UDP for reliable, ordered delivery, using the TCP Cubic implementation in INET [7]. This allows study of the reliability–latency trade-off.
4. **Packet loss and congestion modelling** with bounded queues and random loss channels to stress-test the protocol.
5. **Order prioritisation** — VIP or time-sensitive orders jump the dispatcher queue.
6. **Blockchain-based order verification** [3] to study the added latency of decentralised logging in a smart-city deployment.
7. **Extended statistics**: 95th-percentile delay, per-stage delay breakdown and jitter across a sweep of arrival rates.

8. Conclusions

This project successfully designed, implemented and validated an OMNeT++/INET simulation of a food delivery tracking network. The simulation models the complete five-stage order workflow — OrderRequest, PrepRequest, FoodReady, DeliveryRequest and Delivered — as UDP packets exchanged between four communicating hosts over a switched Ethernet LAN.

The key outcomes are:

1. All five message stages execute in the correct order for every simulated order, confirming protocol correctness.
2. Under General load, 54 of 56 orders (96.4%) were delivered within the 300 s simulation window, with a mean end-to-end delay of ≈ 13.5 s — exactly matching the theoretical prediction.
3. Under HighLoad, the single restaurant–rider chain becomes a bottleneck, demonstrating the model’s ability to capture real queuing dynamics.
4. The Qtenv packet animations (Stages 1–5, Figures 5.1 to 5.5) visually confirm correct packet routing at every step.

The model is simple, modular and fully extensible, providing a solid foundation for the richer features described in Chapter 7: wireless mobility, multiple service nodes, TCP reliability and priority scheduling.

Bibliography

- [1] OpenSim Ltd., “OMNeT++ simulation manual, version 6.x,” <https://omnetpp.org/documentation/>, 2023.
- [2] INET Framework Community, “INET framework — official documentation and API reference,” <https://inet.omnetpp.org/docs/>, 2024.
- [3] U. Tokkozhina, B. M. Mataloto, A. L. Martins, and J. C. Ferreira, “Decentralizing online food delivery services: A blockchain and IoT model for smart cities,” *Mobile Networks and Applications*, vol. 29, pp. 59–69, 2024.
- [4] I. Moumen, N. Rafalia, J. Abouchabaka, and M. Aoufi, “Real-time GPS tracking system for IoT-enabled connected vehicles,” in *E3S Web of Conferences*. EDP Sciences, 2023.
- [5] V. Veselý and J. Zavřel, “Quality control methodology for simulation models of computer network protocols,” arXiv preprint arXiv:2109.12854, 2021.
- [6] S. Manzoor and Y. Yin, “Which simulator to choose for next generation wireless network simulations? NS-3 or OMNeT++,” *Engineering Proceedings*, vol. 46, no. 1, p. 36, 2023.
- [7] I. Angelis, A. Tsipis, E. Christopoulou, and K. Oikonomou, “TCP Cubic implementation in the OMNeT++ INET framework for SIIoT simulation scenarios,” in *CollaborateCom 2023*, ser. LNICST, vol. 561. Springer, Cham, 2024.

A. Appendix

GitHub Repository:

<https://github.com/nishatnishi/omnet>

The complete source code — NED files, all four C++ application modules, `OrderMessage.msg` and `omnetpp.ini` — is available in the repository linked above. The custom message definition is reproduced here for quick reference:

```
1 enum OrderType {
2     ORDER_REQUEST      = 0;
3     PREP_REQUEST       = 1;
4     FOOD_READY         = 2;
5     DELIVERY_REQUEST   = 3;
6     DELIVERED          = 4;
7 };
8
9 class OrderChunk extends inet::FieldsChunk {
10     int      orderId      = 0;
11     int      orderType    = ORDER_REQUEST;
12     string   customerAddress;
13     simtime_t orderTime   = 0;
14 }
```

Listing A.1: `OrderMessage.msg` — complete definition.

Individual Contribution

Table A.1: Individual member contributions.

Member	ID	Contribution
Sanzida Nishat	C231442	Network topology design (NED), TikZ diagrams, literature review and full report writing.
Saima Binte Soyeb	C231449	Custom message type (<code>OrderMessage.msg</code>) and all four C++ application modules (<code>CustomerApp</code> , <code>DispatcherApp</code> , <code>RestaurantApp</code> , <code>RiderApp</code>).
Rahatun Nesa Rahi	C231474	<code>omnetpp.ini</code> configuration, running both simulation configurations, collecting scalar results and result analysis.